

ELIXIR

The Blood Donors Portal

Complete Technical Documentation

How the site works end to end — the technologies, languages, APIs, database, authentication system, and deployment architecture that power it.

Contents

1. What This Project Is
2. Technology Stack At a Glance
3. Languages Used and Where
4. How Everything Connects (Architecture)
5. Frontend, Explained
6. Backend, Explained
7. Database, Explained
8. Authentication System, Explained
9. Complete API Reference
10. Deployment Architecture
11. Key Concepts Glossary

1. What This Project Is

Elixir is a web application that connects blood donors with people who need blood, hospitals, and blood banks. It has three core capabilities: letting visitors search for donors by blood group and city, letting people register themselves as donors, and providing reference information (compatibility charts, emergency helplines). Since the last update, it also has an account system, so only signed-in users can search for or register as donors.

Structurally, it is a normal modern web application split into three independent pieces that talk to each other over the internet: a **frontend** (what visitors see and click on in their browser), a **backend** (a server that handles requests and talks to the database), and a **database** (where donor records, blood requests, and helpline numbers are permanently stored). Each piece is hosted on a different company's servers.

2. Technology Stack At a Glance

Layer	Technology	What it does here
Frontend framework	React 18	Builds the interactive interface as reusable components
Frontend build tool	Vite	Compiles and serves the React code, bundles it for deployment
Frontend styling	Tailwind CSS	Utility-based styling system for all colors, spacing, layout
Frontend routing	React Router	Switches between pages (Home, Find Donors, etc.) without a full page reload
Backend framework	Express.js	Handles incoming web requests and defines the API
Backend runtime	Node.js	Runs the JavaScript backend code on the server
Database	PostgreSQL	Stores donors, blood requests, and helpline data permanently
Database + Auth host	Supabase	Hosts the Postgres database and handles user login/signup
Backend hosting	Render	Runs the Express server continuously, 24/7
Frontend hosting	Vercel	Serves the built React app to visitors worldwide
Version control	Git + GitHub	Tracks code changes and stores the project's source code

*This is sometimes called a **three-tier architecture**: presentation tier (frontend), logic tier (backend), and data tier (database) — each replaceable independently.*

3. Languages Used and Where

Language	Used in	Purpose
JavaScript (as JSX)	All frontend files (.jsx)	Describes the UI — React components mix JavaScript logic with HTML-like markup (JSX)
JavaScript	All backend files (.js)	Defines API routes, database queries, and server logic
SQL	backend/db/schema.sql	Defines the database's tables, columns, and relationships
CSS (via Tailwind)	Every component's className props	Utility classes like text-wine, px-6 style every element
HTML	frontend/index.html	The single page shell React mounts itself into
JSON	package.json files, API responses	Structured data format used for configuration and all API data exchange

Notice the same language — JavaScript — is used for both frontend and backend. This is common in modern web development (sometimes called a "JavaScript full-stack") and is why both frontend/ and backend/ have their own package.json and node_modules — they're two separate JavaScript projects that happen to use the same language.

4. How Everything Connects (Architecture)

Here is what happens, step by step, when someone searches for a donor:

- **1. Browser loads the site** — Vercel serves the pre-built React app (HTML, CSS, JS files) to the visitor's browser.
- **2. User navigates to Find Donors** — React Router swaps the visible page instantly, no server involved yet.
- **3. Page requests data** — the React component calls a function in `api.js`, which sends an HTTP request to the Render backend's URL, e.g. `https://elixir-portal.onrender.com/api/donors`.
- **4. Request includes a login token** — since this route is protected, the request carries the user's Supabase session token in its Authorization header.
- **5. Backend verifies the token** — Express middleware checks the token is valid and not expired by asking Supabase's auth service to confirm it.
- **6. Backend queries the database** — if the token is valid, the route handler runs a SQL query against the donors table, connecting to Supabase's Postgres database over an encrypted connection.
- **7. Data flows back** — Postgres returns rows → Express formats them as JSON → the browser receives them → React renders them as donor cards on the page.

The important idea: the frontend never talks to the database directly. It only ever talks to the backend, and the backend is the only thing with database credentials. This is a standard security boundary — credentials never reach the browser, where anyone could see them.

5. Frontend, Explained

Folder structure (inside `frontend/src/`)

File/folder	Purpose
<code>main.jsx</code>	Entry point — mounts the React app into <code>index.html</code> and wraps it in the router
<code>App.jsx</code>	Defines every page's URL (route) and which pages require login
<code>pages/</code>	One file per page — Home, FindDonors, BloodInfo, Helpline, RegisterDonor, Login
<code>components/</code>	Reusable pieces used across multiple pages — Navbar, Footer, ProtectedRoute
<code>context/AuthContext.jsx</code>	Tracks whether a user is logged in, shared across the whole app
<code>api.js</code>	Every function that talks to the backend lives here, in one place
<code>supabaseClient.js</code>	Connects the frontend to Supabase's authentication service

Components vs. Pages

A **page** (in pages/) represents a full screen — something with its own URL, like /helpline. A **component** (in components/) is a smaller reusable piece that appears across multiple pages, like the navigation bar at the top or the footer at the bottom. Pages are usually built out of several components plus their own unique content.

State and Context

React components remember information using `useState` — for example, `FindDonors.jsx` uses state to remember what's currently typed in the search box. Some information needs to be known by many different components at once — like "is anyone logged in right now?" For that, this project uses React's **Context** feature (`AuthContext.jsx`), which makes the current user available to any component that asks for it, without having to manually pass it down through every layer.

Styling with Tailwind

Instead of writing separate CSS files, elements are styled with utility class names directly in the markup, for example `className="bg-wine text-paper px-6 py-3"`. Each class does one small thing (background color, text color, padding) and they combine to build the full look. The custom color names (wine, amber, teal, paper, ink) are defined once in `tailwind.config.js`.

6. Backend, Explained

Folder structure (inside backend/)

File/folder	Purpose
server.js	Starts the Express server and connects every route to its URL path
routes/donors.js	Handles searching for and registering donors
routes/requests.js	Handles blood requests (built, not yet used by the UI)
routes/helplines.js	Returns emergency contact numbers
middleware/requireAuth.js	Checks a request's login token before letting it through
db/db.js	Opens and manages the connection to the Postgres database
db/schema.sql	The database's table definitions — run once to set it up
supabaseAdmin.js	Lets the backend verify login tokens with Supabase

What is Express, really?

Express is a library that makes it simple to say "when a request arrives at this URL with this method, run this function." For example:

```
app.use('/api/donors', requireAuth, donorsRouter)
```

This line means: any request to a URL starting with `/api/donors` first passes through `requireAuth` (which checks login), and if that passes, it's handed to `donorsRouter`, which contains the actual logic for searching or creating donor records.

Middleware

Middleware is a function that runs *in between* a request arriving and a response being sent — it can inspect, modify, block, or pass along the request. This project uses three: `cors()` (allows the frontend's domain to make requests to the backend's different domain), `express.json()` (parses incoming JSON request bodies), and the custom `requireAuth` (blocks requests without a valid login token).

Connecting to the database

The backend uses a library called `pg` ("node-postgres") to talk to PostgreSQL. Rather than opening a new connection for every single request (slow), it keeps a small reusable "pool" of open connections ready to go, defined once in `db.js` and imported wherever a route needs to run a query.

7. Database, Explained

The database has three tables, defined in `backend/db/schema.sql` and physically stored on Supabase's servers.

donors

Column	Type	Meaning
id	number	Unique identifier, assigned automatically
name	text	Donor's full name
age	number	Must be between 18 and 65
blood_group	enum	One of: A+ A- B+ B- AB+ AB- O+ O-
city	text	Used for location-based search
phone	text	Contact number
email	text	Optional
last_donation_date	date	Optional, for eligibility tracking
is_available	true/false	Whether they currently show up in search results
created_at	timestamp	Set automatically when the row is created

blood_requests

Holds requests for blood from patients (patient name, blood group needed, city, hospital, contact number, and a status of open/fulfilled/cancelled). The API for this table is built and working, but no page in the UI uses it yet — it's ready for a future "request blood" feature.

helplines

A short table of emergency contact numbers shown on the Helpline page — name, number, and a short note. Editing rows here through Supabase's Table Editor updates the live site immediately, with no code change or redeploy needed.

Why Supabase specifically?

Supabase is a company that hosts a real PostgreSQL database for free (on a limited tier) and adds convenient extras on top — a visual table editor, an authentication system, and instant REST/GraphQL APIs. This project uses Supabase for two things: hosting the Postgres database itself, and handling user login/signup (Supabase Auth). The backend connects to it using a **Session Pooler** connection — a connection method Supabase provides specifically for servers (like Render) that only support IPv4 networking, as opposed to the "direct" connection which requires IPv6.

8. Authentication System, Explained

Elixir uses **Supabase Auth** rather than a custom-built login system. Supabase handles the hard, security-sensitive parts (password hashing, email confirmation, session expiry) so the app just asks Supabase questions like "is this person logged in?" and "is this token valid?"

What is a JWT?

When someone logs in, Supabase gives their browser a **JWT** (JSON Web Token) — a long, encoded string that proves who they are without needing to send their password again on every request. It's like a wristband at an event: once checked at the door, you can show the wristband instead of your ID every time. The token has an expiry time and is signed cryptographically, so it can't be faked or edited by the person holding it.

How login works, step by step

- User submits the login form on the Login page.
- `supabaseClient.js` sends the email/password directly to Supabase's authentication servers (not to Elixir's own backend).
- Supabase verifies the password and, if correct, returns a JWT session token to the browser.
- `AuthContext.jsx` stores this in memory and makes the logged-in user available everywhere via `useAuth()`.
- `ProtectedRoute.jsx` checks this before showing Find Donors or Register — no user means an automatic redirect to `/login`.

How the backend double-checks it

A frontend-only check can be bypassed by a technical user calling the API directly. So every request to `/api/donors` also carries the JWT in its Authorization header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

The `requireAuth` middleware takes that token and asks Supabase (using a separate, more powerful "service role" key that only the backend knows) whether it's genuinely valid. Only if Supabase confirms it does the request continue to the actual donor-search logic. This means the API itself is protected, not just the page around it.

Two different Supabase keys — why?

Key	Lives in	Can do
anon (public) key	Frontend (<code>VITE_SUPABASE_ANON_KEY</code>)	Sign up, sign in — safe to expose in browser code
service_role key	Backend only (<code>SUPABASE_SERVICE_ROLE_KEY</code>)	Full admin access, including verifying any user's token — must stay secret

9. Complete API Reference

All endpoints are prefixed with the backend's base URL, e.g. <https://elixir-portal.onrender.com>. Every response is JSON.

Method & Path	Auth?	Purpose	Request body / params
GET /api/health	No	Confirms the server is running	—
GET /api/donors	Yes	Search donors by blood group and/or city	?blood_group=0+&city;=Noida (query params)
POST /api/donors	Yes	Register a new donor	{ name, age, blood_group, city, phone, email, last_donation_date }
GET /api/requests	No	List blood requests, optionally by status	?status=open (query param)
POST /api/requests	No	Create a new blood request	{ patient_name, blood_group, city, units_needed, hospital, contact_phone }
GET /api/helplines	No	Get emergency contact numbers	—

"Auth?" means the request must include a valid Authorization: Bearer <token> header or it's rejected with a 401 Unauthorized response before any database query runs.

Example — a successful donor search response

```
{
  "donors": [
    {
      "id": 1,
      "name": "Akshat Vedant",
      "age": 24,
      "blood_group": "B+",
      "city": "Greater Noida",
      "phone": "9450445116",
      "last_donation_date": null
    }
  ]
}
```

Example — a rejected request (no login token)

```
{  
  "error": "Sign in required."  
}
```

10. Deployment Architecture

Three separate companies host three separate pieces of this project. None of them know about the others directly — they're connected only through URLs and environment variables you configured.

Service	Hosts	Connects via
Vercel	The built frontend (static HTML/CSS/JS)	Auto-deploys on every git push to GitHub
Render	The Express backend, running continuously	Auto-deploys on every git push to GitHub
Supabase	The Postgres database and auth system	Backend connects via DATABASE_URL; frontend connects via Supabase's public API
GitHub	The source code itself	Both Vercel and Render watch this repository for changes

Environment variables — what each one does

Variable	Set where	Purpose
DATABASE_URL	Render (backend)	Tells the backend how to connect to the Postgres database
SUPABASE_URL	Render (backend)	Base URL for verifying login tokens
SUPABASE_SERVICE_ROLE_KEY	Render (backend)	Secret admin key used to verify any user's token
VITE_API_URL	Vercel (frontend)	Tells the frontend where the live backend is
VITE_SUPABASE_URL	Vercel (frontend)	Lets the frontend talk to Supabase Auth directly
VITE_SUPABASE_ANON_KEY	Vercel (frontend)	Public key allowing sign-up/sign-in from the browser

Anything prefixed VITE_ is bundled into the frontend's code and visible to anyone who opens their browser's developer tools — this is fine for URLs and public keys, but is exactly why the service role key and database password never get that prefix.

Why a frontend rewrite rule (vercel.json) was needed

React Router handles page navigation entirely inside the browser — Vercel's server has no record of a page called /register, since only index.html physically exists as a file. vercel.json tells Vercel: for any URL that doesn't match a real file, serve index.html anyway and let React Router take over from there. This is standard for any "single-page application" (SPA).

11. Key Concepts Glossary

API (Application Programming Interface)

A defined set of URLs a program can send requests to in order to get or change data — in this project, everything under /api/...

REST API

A common style of API where each URL represents a "thing" (like /donors) and the HTTP method (GET, POST) says what to do with it.

JSON

A lightweight text format for structured data, e.g. {"name": "Akshat"} — used for almost all data sent between frontend and backend.

JWT (JSON Web Token)

A signed, tamper-proof string that proves a user is logged in, without needing to re-check their password every time.

CORS (Cross-Origin Resource Sharing)

A browser security rule that blocks a webpage from one domain (Vercel) from freely calling an API on another domain (Render) unless the API explicitly allows it — hence cors() in server.js.

SSL/TLS

Encryption for data traveling over the internet. Supabase requires it for database connections, which is why db.js includes an ssl option.

Environment variable

A setting (like a password or URL) kept outside the code itself, so it can differ between your laptop, GitHub, and the live servers without ever being committed to source control.

Git / GitHub

Git tracks every change made to the code over time; GitHub is a website that stores that history online and lets Vercel/Render watch for new changes.

Connection pooling

Reusing a small set of open database connections instead of opening a brand new one for every single request — faster and more efficient.

IPv4 vs IPv6

Two different addressing systems for reaching servers on the internet. Render's free tier only supports outbound IPv4, which is why the project uses Supabase's IPv4-compatible "Session Pooler" instead of its IPv6-only "Direct connection."

SPA (Single-Page Application)

A website that loads one HTML page once, then swaps content in and out with JavaScript instead of asking the server for a brand-new page each time you click a link.

Middleware

Code that runs in between a request arriving and a response being sent, able to inspect, modify, or block it — used here for CORS, JSON parsing, and login checks.

This document reflects the project as built: a React + Vite frontend on Vercel, an Express backend on Render, and a PostgreSQL database with built-in authentication on Supabase, all connected through a REST API secured with Supabase-issued JWT tokens.